Desk Overlord: Hackathon Playbook

Oct 24, 2026 · @Justin

The plan in one minute

Desk Overlord is a creature on your desk that turns, flashes and beeps when your laptop is on a distraction, and calms down when you get back to work. A Python script on the laptop watches the active window; the Arduino is the creature's body.

How it works

1. `overlord.py` reads the title of the window you are looking at twice a second and checks it against a keyword list (YouTube, Reddit, Netflix and so on).
2. An anger score from 0 to 100 rises while you are distracted and falls while you work. Thresholds turn it into four moods: calm, watching, angry, furious.
3. The laptop sends one line over USB, such as `2|YOUTUBE AGAIN?`. The Arduino turns the servo head, sets the RGB eyes, drives the buzzer and writes the LCD.
4. A button on the creature is a beg-for-mercy button. It sends `P` back to the laptop, which grants a short truce.

Why it holds up in front of judges

- Everything runs on the laptop. No cloud, no account, and window titles are never stored.
- It is rule-based on purpose: explainable, works offline, cannot hallucinate. Describe it that way, not as AI.
- The link is two-way with a watchdog: if the laptop goes quiet for 5 seconds, the creature shows NO SIGNAL and glows blue.
- A session report (CSV and chart) turns the gimmick into data.

Milestones

Milestone	Time	What must be true
M1	1:15 pm	Opening YouTube makes the servo turn. Record a 30-second safety video.

Milestone	Time	What must be true
M2	3:30 pm	All four moods, LCD taunts and the truce button work on the finished body.
M3	5:50 pm	Video uploaded, and the upload checked from another device.

Ask an organizer now: the exact submission deadline, the maximum video length, and where videos are uploaded. This plan assumes everything is due at 6:30 pm and aims to submit by 5:50 pm.

Rules of the day

- Ugly and working beats pretty and broken. Polish only after M1.
- Change one thing, then test it.
- At every milestone, photograph the wiring and copy the code to a shared folder.
- Feature freeze at 4:30 pm. After that you fix bugs and record, nothing else.

Roles

Each person owns one lane, and one HP laptop (the host) runs the code and holds the Arduino. The other two laptops are for slides, notes and video editing.

Person	Owns	Hands off
A: Hardware	Wiring, self-test, servo angles, LCD, the cardboard body	Tells B the COM port and which parts pass the self-test
B: Python	Setup on the host laptop, `overlord.py`, keyword list, timing, session report	Tells A when levels 0 to 3 are being sent correctly
C: Character and story	Taunt lines, creature design, pitch, video script, filming, upload	Tells A and B by 2:30 pm what the video needs to show

Working rules

- Swap roles if someone is stuck for 20 minutes. Fresh eyes find wiring mistakes fast.
- Only the host laptop is plugged into the Arduino. The other two work on copies of the code in a shared folder.

- C also keeps the clock: calls the milestones at 1:15, 3:30 and 4:30 pm, and says what to cut if the team is behind (see the schedule).
- At least two people must be able to run the demo from a cold start. Put the start-up steps on a sticky note.

Hour-by-hour schedule

The working demo comes first (M1 at 1:15 pm), the polish second, and nothing new after the 4:30 pm freeze. The blocks add up to 7 hours 45 minutes, 10:45 am to 6:30 pm.

Time	Block	A: Hardware	B: Python	C: Character and story	Done when
10:45-11:15	0. Setup	Unpack the kit, count parts against the list, set up the breadboard power rails	Install Arduino IDE 2 and pyserial, find the COM port, upload Blink	Name the creature, ask the organizer the three questions, gather cardboard, tape and glue	Blink uploads and the Uno shows up as a COM port
11:15-12:15	1. Parts on the bench	Wire servo, RGB LED, buzzer, button, one at a time, running the self-test after each	Save `overlord.py`, run it with `--dry-run`, watch the level climb on YouTube	Write 10 taunt lines, sketch the creature, collect body materials	Self-test passes for all four parts, and dry-run reaches level 3 on YouTube
12:15-1:15	2. LCD and link (lunch in shifts)	Wire the LCD (contrast pot first), upload the full sketch	Test the sketch from Serial Monitor, then run `overlord.py` against the Arduino	Draft the video storyboard, eat in shifts so someone always guards the build	**M1:** opening YouTube turns the servo. Record a 30-second safety video
1:15-2:30	3. Full Overlord	Calibrate the `ANGLE_` values, fix the parts to a rigid base, test the button	Tune `LEVEL_AT`, gain and decay, test the truce, add keywords	Finalize taunts, start the cardboard head and eyes	Four moods look and sound distinct, and the button grants a truce
2:30-3:30	4. Body	Mount servo, LCD and LED in the creature, tidy and tape down wires	Run a 10-minute test while doing real work, fix crashes, check the session report	Finish the body, write the pitch script, set up the filming spot	**M2:** full demo works on the finished body 3 times in a row

Time	Block	A: Hardware	B: Python	C: Character and story	Done when
3:30-4:30	5. Test and polish	Re-seat every jumper, photograph the wiring, check spares	Turn off notifications and sleep, write the run steps on a sticky note	Rehearse the 90-second demo with a timer, lock the script	Demo works 5 times in a row, script locked
4:30-5:30	6. Record	Operate the creature and hold the camera	Run the laptop and screen recording	Direct, narrate, pick the best take	Best take saved in two places
5:30-5:50	7. Submit	Keep the demo ready to run live	Upload the video, fill in the form	Proofread the text	**M3:** the link opens on another device
5:50-6:30	Buffer	Stay ready	Retry failed uploads	Rehearse judge questions	Fixes only, nothing new

If you fall behind: at each milestone, if you are more than 30 minutes late, cut the next item below instead of trying to catch up.

1. The session report chart (keep the CSV).
2. The truce button.
3. LCD taunts (keep the fixed mood text).
4. Buzzer escalation (keep a single beep).

Never cut: the servo and RGB eyes driven by Python, the safety video at M1, and the 4:30 pm freeze.

Parts and wiring

You wire the creature in five stages and run the self-test after each, so a mistake is found while only one part is new. Everything below is in your Elegoo kit except the cardboard, tape and glue.

Parts checklist

- [ ] Arduino UNO R3 and its USB cable
- [ ] 830-point breadboard
- [ ] SG90 servo (the kit has only one, so the head is the only moving part)
- [ ] RGB LED (the creature's eyes)

- [ ] 5 resistors of 220 ohm, bands red-red-brown (3 for the LED, 1 for the buzzer, 1 for the LCD backlight; 330 ohm also works)
- [ ] Active buzzer (the black one with the sticker on top; peel the sticker off)
- [ ] 1 push button
- [ ] LCD1602 with pin header
- [ ] 10K potentiometer (LCD contrast)
- [ ] Male-to-male jumper wires (the servo's three-hole plug takes one wire per hole)
- [ ] 100 uF electrolytic capacitor, only if the servo makes the board reset
- [ ] From the organizers: cardboard, tape, hot glue, markers, googly eyes

Before you wire anything: unplug the USB cable whenever you change wires. Never connect 5V straight to GND. Do not use pins 0 and 1, which the USB link needs.

Stage 0: power rails. Uno 5V to the breadboard's red rail, Uno GND to the blue rail.

Stage 1: servo (self-test: the head swings left, right, centre)

1. Orange wire (signal) to D9.
2. Red wire to the 5V rail.
3. Brown wire to the GND rail.

Stage 2: RGB LED (self-test: red, green, blue in turn). Hold the LED with its longest leg second from the left. The legs from the left are red, the longest one (common), green, blue.

1. Red leg through a 220 ohm resistor to D3.
2. Green leg through a 220 ohm resistor to D5.
3. Blue leg through a 220 ohm resistor to D6.
4. Longest leg to the GND rail.

Stage 3: buzzer (self-test: one beep)

1. The leg marked + through a 220 ohm resistor to D10.
2. The other leg to the GND rail.

Stage 4: button (self-test: the LCD or Serial Monitor shows YES while you hold it). Place it across the centre gap of the breadboard.

1. One side of the button to A0.
2. The other side to the GND rail.
3. If it reads pressed all the time, turn the button 90 degrees.

Stage 5: LCD1602 (the most wires, so it goes last). Plug the LCD's 16-pin header into the breadboard so each pin has its own column, then add jumpers in the same columns. Pin 1 is the end marked 1 or VSS on the board. Wire the potentiometer first: one outer leg to 5V, the other outer leg to GND, the middle leg to LCD pin 3.

LCD pin	Name	Connect to
1	VSS	GND rail
2	VDD	5V rail
3	V0	Middle leg of the potentiometer
4	RS	D12
5	RW	GND rail
6	E	D11
7 to 10	D0 to D3	Leave empty
11	D4	D8
12	D5	D7
13	D6	D4
14	D7	D2
15	A	220 ohm resistor to the 5V rail
16	K	GND rail

Turn the potentiometer slowly until the LCD shows two rows of blocks or text. Then photograph the finished wiring.

Code

Three files do everything: a self-test sketch, the Overlord sketch for the Arduino, and `overlord.py` for the laptop. The Python logic and the Arduino sketch's logic were run against simulated input (a fake clock, fake window titles, a fake serial port and fake pins) and passed. Nothing has run on your actual Arduino or Windows laptop yet, so expect to adjust servo angles and re-check wiring.

Set up the host laptop (about 15 minutes)

1. Install Arduino IDE 2 from arduino.cc if it is not installed. Plug in the Uno, then use the board dropdown at the top to pick Arduino Uno and its COM port.
2. Open PowerShell and run `py --version`. Any Python 3.8 or newer is fine.
3. Run `py -m pip install pyserial matplotlib`. Install `pyserial`, not `serial`, which is a different package.
4. Make a folder such as `Documents\overlord` and save `overlord.py` in it. In the Arduino IDE, save each sketch under the name given below, which creates a folder of the same name.

File 1: overlord_selftest

Upload this after each wiring stage. Whatever does not happen points at the stage you just wired. The same messages also appear in Serial Monitor at 9600 baud.

```
// SELF-TEST: runs each part one at a time, forever.
// Upload it after wiring each stage and note what does NOT happen.
// The same messages also appear in Serial Monitor (9600 baud).

#include <Servo.h>
#include <LiquidCrystal.h>

const byte PIN_SERVO  = 9;
const byte PIN_R      = 3;
const byte PIN_G      = 5;
const byte PIN_B      = 6;
const byte PIN_BUZZER = 10;
const byte PIN_BUTTON = A0;
const bool COMMON_ANODE = false;         // set true if the colours look
inverted
LiquidCrystal lcd(12, 11, 8, 7, 4, 2);   // RS, E, D4, D5, D6, D7
Servo head;

void setRGB(byte r, byte g, byte b) {
  if (COMMON_ANODE) { r = 255 - r; g = 255 - g; b = 255 - b; }
  analogWrite(PIN_R, r);
  analogWrite(PIN_G, g);
  analogWrite(PIN_B, b);
}

void say(const char* what) {
  lcd.setCursor(0, 1);
```

```
  lcd.print("                ");
  lcd.setCursor(0, 1);
  lcd.print(what);
  Serial.println(what);
}

void setup() {
  Serial.begin(9600);
  pinMode(PIN_R, OUTPUT);
  pinMode(PIN_G, OUTPUT);
  pinMode(PIN_B, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  lcd.begin(16, 2);
  lcd.print("SELF TEST");
  head.attach(PIN_SERVO);
  head.write(80);
}

void loop() {
  say("servo");
  head.write(40);  delay(700);
  head.write(120); delay(700);
  head.write(80);  delay(500);

  say("rgb red");   setRGB(255, 0, 0); delay(600);
  say("rgb green"); setRGB(0, 255, 0); delay(600);
  say("rgb blue");  setRGB(0, 0, 255); delay(600);
  setRGB(0, 0, 0);

  say("buzzer");
  digitalWrite(PIN_BUZZER, HIGH); delay(200);
  digitalWrite(PIN_BUZZER, LOW);  delay(300);

  say("press button");
  unsigned long until = millis() + 3000;
  while (millis() < until) {
    lcd.setCursor(13, 1);
    lcd.print(digitalRead(PIN_BUTTON) == LOW ? "YES" : "no ");
  }
  say("--- again ---");
  delay(500);
}
```

File 2: overlord (the creature)

Upload this once the LCD is wired. Everything you will want to change is in the Tuning block near the top. To calibrate, set `ANGLE_YOU` until the face points at the person's seat, then keep `ANGLE_AWAY` about 80 degrees from it and `ANGLE_MID` halfway.

Level	Meaning	Head	Eyes	Sound
-1	No signal	Faces away	Blue	None
0	Calm	Faces away	Soft green	None
1	Watching	Middle angle	Amber	None
2	Angry	Glares at you, shakes a little	Orange-red	One chirp every 2 seconds
3	Furious	Glares at you, shakes hard	Flashing red	Rapid beeping

```
// DESK OVERLORD - Arduino side
// Receives lines like "2|YOUTUBE AGAIN?" over USB (9600 baud) and shows a
mood.
// Sends "P" back when the button is pressed (a request for a truce).

#include <Servo.h>
#include <LiquidCrystal.h>

// ---------- Pins (must match the wiring section) ----------
const byte PIN_SERVO  = 9;
const byte PIN_R      = 3;    // PWM
const byte PIN_G      = 5;    // PWM
const byte PIN_B      = 6;    // PWM
const byte PIN_BUZZER = 10;
const byte PIN_BUTTON = A0;
LiquidCrystal lcd(12, 11, 8, 7, 4, 2);   // RS, E, D4, D5, D6, D7

// ---------- Tuning: edit these, not the logic ----------
const bool COMMON_ANODE = false;   // true if the LED colours look inverted
const bool SOUND_ON     = true;    // false = silent rehearsal
const int  ANGLE_AWAY   = 40;      // head angle when ignoring you
const int  ANGLE_MID    = 80;      // head angle when watching you
const int  ANGLE_YOU    = 120;     // head angle when glaring at you
```

```
const unsigned long LINK_TIMEOUT_MS = 5000;   // silence this long = NO
SIGNAL

Servo head;
int  level = -1;                   // -1 = no link, 0..3 = mood
char line2[17] = "WAITING FOR PC";
bool lcdDirty = true;
int  currentAngle = ANGLE_AWAY;
unsigned long lastMsgMs = 0;
unsigned long lastServoMs = 0;
char buf[24];
byte bufLen = 0;
bool lastButton = HIGH;
unsigned long lastButtonMs = 0;

void setRGB(byte r, byte g, byte b) {
  if (COMMON_ANODE) { r = 255 - r; g = 255 - g; b = 255 - b; }
  analogWrite(PIN_R, r);
  analogWrite(PIN_G, g);
  analogWrite(PIN_B, b);
}

// Write exactly 16 characters to one LCD row (pads with spaces, no flicker).
void lcdLine(byte row, const char* text) {
  lcd.setCursor(0, row);
  byte i = 0;
  while (text[i] != 0 && i < 16) { lcd.write(text[i]); i++; }
  while (i < 16) { lcd.write(' '); i++; }
}

void setLevel(int newLevel, const char* msg) {
  if (newLevel != level || strncmp(line2, msg, 16) != 0) lcdDirty = true;
  level = newLevel;
  strncpy(line2, msg, 16);
  line2[16] = 0;
}

// ---------- Input from the laptop: "<level>|<text>\n" ----------
void readSerial(unsigned long now) {
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '\r') continue;
    if (c == '\n') {
```

```
      buf[bufLen] = 0;
      if (bufLen >= 1 && buf[0] >= '0' && buf[0] <= '3') {
        const char* msg = (bufLen >= 2 && buf[1] == '|') ? buf + 2 : "";
        setLevel(buf[0] - '0', msg);
        lastMsgMs = now;
      }
      bufLen = 0;
    } else if (bufLen < sizeof(buf) - 1) {
      buf[bufLen++] = c;
    }
  }
}

// ---------- Button: send "P" once per press ----------
void checkButton(unsigned long now) {
  bool b = digitalRead(PIN_BUTTON);
  if (b != lastButton && now - lastButtonMs > 40) {
    lastButtonMs = now;
    lastButton = b;
    if (b == LOW) Serial.println("P");
  }
}

// ---------- Outputs ----------
void updateLCD() {
  if (!lcdDirty) return;
  lcdDirty = false;
  const char* top = "NO SIGNAL";
  if (level == 0) top = "ALL IS CALM";
  if (level == 1) top = "I SEE YOU...";
  if (level == 2) top = "CLOSE THAT TAB";
  if (level == 3) top = "ENOUGH!!!";
  lcdLine(0, top);
  lcdLine(1, line2);
}

void updateServo(unsigned long now) {
  if (now - lastServoMs < 20) return;
  lastServoMs = now;
  int target = ANGLE_AWAY;
  int shake = 0;
  if (level == 1) target = ANGLE_MID;
  if (level == 2) { target = ANGLE_YOU; shake = 6; }
```

```
  if (level == 3) { target = ANGLE_YOU; shake = 14; }
  if (shake > 0) target += ((now / 120) % 2 == 0) ? shake : -shake;
  target = constrain(target, 10, 170);
  if (currentAngle < target) currentAngle = min(currentAngle + 4, target);
  else if (currentAngle > target) currentAngle = max(currentAngle - 4,
target);
  head.write(currentAngle);
}

void updateLED(unsigned long now) {
  if (level == 0) setRGB(0, 40, 0);                  // calm: soft green
  else if (level == 1) setRGB(120, 70, 0);           // watching: amber
  else if (level == 2) setRGB(255, 40, 0);           // angry: orange-red
  else if (level == 3) {                             // furious: flashing red
    if ((now / 150) % 2 == 0) setRGB(255, 0, 0); else setRGB(0, 0, 0);
  }
  else setRGB(0, 0, 60);                             // no signal: blue
}

void updateBuzzer(unsigned long now) {
  bool on = false;
  if (SOUND_ON) {
    if (level == 2) on = (now % 2000) < 90;          // short chirp every 2 s
    if (level == 3) on = (now % 400) < 200;          // rapid beeping
  }
  digitalWrite(PIN_BUZZER, on ? HIGH : LOW);
}

void setup() {
  Serial.begin(9600);
  pinMode(PIN_R, OUTPUT);
  pinMode(PIN_G, OUTPUT);
  pinMode(PIN_B, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  lcd.begin(16, 2);
  head.attach(PIN_SERVO);
  head.write(currentAngle);
}

void loop() {
  unsigned long now = millis();
  readSerial(now);
```

```
  if (level >= 0 && now - lastMsgMs > LINK_TIMEOUT_MS) setLevel(-1, "CHECK
PYTHON");
  checkButton(now);
  updateLCD();
  updateServo(now);
  updateLED(now);
  updateBuzzer(now);
}
```

Test the creature without Python. Upload the sketch, open Serial Monitor at 9600 baud, set the line-ending dropdown to Newline, and send `2|HELLO`. The LCD should show CLOSE THAT TAB over HELLO, the eyes turn orange and the head turns toward you. Send `0|CALM` to relax it. After 5 seconds of silence it falls back to NO SIGNAL with blue eyes, which is the watchdog working. Close Serial Monitor before running Python, because only one program can hold the port.

File 3: overlord.py (the laptop)

Save this as `overlord.py` in your `overlord` folder. It needs only `pyserial` for the Arduino link (and `matplotlib` for the report chart). Window detection uses Windows itself, so nothing else is installed.

```
"""Desk Overlord - laptop side (Windows).

Reads the title of the window you are looking at, decides whether it is a
distraction, keeps an anger score, and tells the Arduino which mood to show.
Everything runs on this laptop. Nothing goes to the internet.

Setup:  py -m pip install pyserial matplotlib
Test:   py overlord.py --dry-run        (no Arduino needed)
Run:    py overlord.py                  (finds the Arduino by itself)
        py overlord.py --port COM5      (if it cannot find it)
        py overlord.py --real           (slow, realistic timing)
Quit:   Ctrl+C  (saves session.csv and session_report.png)
"""
import argparse
import csv
import ctypes
import math
import time

try:
```

```
    import serial
    from serial.tools import list_ports
except ImportError:
    serial = None

# ---------- Edit these ----------
BAUD = 9600
TICK = 0.5   # seconds between checks

# keyword in the window title (lowercase) -> name shown on the LCD (9 letters
max)
DISTRACTIONS = {
    "youtube": "YOUTUBE", "reddit": "REDDIT", "netflix": "NETFLIX",
    "tiktok": "TIKTOK", "instagram": "INSTAGRAM", "twitch": "TWITCH",
    "facebook": "FACEBOOK", "discord": "DISCORD", "9gag": "9GAG",
}

LEVEL_AT = (10, 30, 60)   # anger needed for level 1, 2, 3 (anger tops out at
100)

# (anger gained per second when distracted, lost per second when focused,
truce seconds)
DEMO_TIMING = (4.0, 6.0, 15)    # fast: level 3 after 15 s. Good for the
video
REAL_TIMING = (1.0, 2.0, 300)   # slow: level 3 after 60 s. For real use


# ---------- Window detection (Windows only, no extra packages) ----------
def active_window_title():
    if not hasattr(ctypes, "windll"):
        return ""
    user32 = ctypes.windll.user32
    hwnd = user32.GetForegroundWindow()
    length = user32.GetWindowTextLengthW(hwnd)
    buf = ctypes.create_unicode_buffer(length + 1)
    user32.GetWindowTextW(hwnd, buf, length + 1)
    return buf.value


def match_distraction(title):
    t = title.lower()
    for keyword, name in DISTRACTIONS.items():
        if keyword in t:
```

```
            return name
    return None


# ---------- Mood logic ----------
def level_for(anger):
    return sum(1 for limit in LEVEL_AT if anger >= limit)


def lcd_text(level, site, truce_left):
    """Second LCD line: ASCII only, 16 characters max. Edit the taunts
here."""
    if truce_left > 0:
        text = f"TRUCE {truce_left}s"
    elif level == 0:
        text = "GOOD HUMAN"
    elif site is None:
        text = "STILL WATCHING"
    elif level == 1:
        text = f"{site}?"
    elif level == 2:
        text = f"{site} AGAIN?"
    else:
        text = "CLOSE IT. NOW."
    return text.encode("ascii", "ignore").decode()[:16]


# ---------- Serial link ----------
def find_arduino_port():
    for p in list_ports.comports():
        info = f"{p.description} {p.manufacturer}".lower()
        if "arduino" in info or "ch340" in info or "usb serial" in info:
            return p.device
    return None


def open_port(port_arg):
    while True:
        port = find_arduino_port() if port_arg == "auto" else port_arg
        if port:
            try:
                ser = serial.Serial(port, BAUD, timeout=0.05)
                time.sleep(2.5)   # the Uno restarts when the port opens
```

```
                print(f"Connected on {port}")
                return ser
            except (serial.SerialException, OSError) as err:
                print(f"Could not open {port}: {err}")
                print("  Close Serial Monitor and any other program using the
port.")
        else:
            seen = [f"{p.device} ({p.description})" for p in
list_ports.comports()]
            print("No Arduino found. Ports I can see:", seen or "none")
        print("Retrying in 2 s (Ctrl+C to quit)...")
        time.sleep(2)


def exchange(ser, message):
    """Send one line to the Arduino. Returns True if its button was
pressed."""
    ser.write(message.encode("ascii"))
    pressed = False
    while ser.in_waiting:
        if ser.readline().decode("ascii", "ignore").strip() == "P":
            pressed = True
    return pressed


# ---------- Session report (titles are never saved) ----------
def save_report(log):
    if len(log) < 2:
        return
    with open("session.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["seconds", "anger", "on_distraction"])
        writer.writerows(log)
    pct = 100.0 * sum(1 for row in log if row[2]) / len(log)
    print(f"Session: {log[-1][0]:.0f} s, {pct:.0f}% of checks were on a
distraction. Saved session.csv")
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        print("For a chart too: py -m pip install matplotlib")
        return
```

```
    t = [row[0] for row in log]
    fig, ax = plt.subplots(figsize=(9, 3.5))
    ax.fill_between(t, 0, 100, where=[row[2] for row in log], step="post",
                    alpha=0.2, label="on a distraction")
    ax.plot(t, [row[1] for row in log], linewidth=2, label="Overlord anger")
    ax.set_ylim(0, 100)
    ax.set_xlabel("seconds")
    ax.set_ylabel("anger (0-100)")
    ax.set_title(f"{pct:.0f}% of this session was spent on distractions")
    ax.legend(loc="upper right")
    fig.tight_layout()
    fig.savefig("session_report.png", dpi=150)
    print("Saved session_report.png")


# ---------- Main loop ----------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--port", default="auto", help="e.g. COM5 (default: find
it)")
    ap.add_argument("--dry-run", action="store_true",
                    help="no Arduino: print what would be sent (also prints
window titles)")
    ap.add_argument("--real", action="store_true", help="slow, realistic
timing")
    args = ap.parse_args()
    gain, decay, truce_len = REAL_TIMING if args.real else DEMO_TIMING

    ser = None
    if not args.dry_run:
        if serial is None:
            raise SystemExit("pyserial is missing. Run:  py -m pip install
pyserial")
        ser = open_port(args.port)

    anger, truce_until = 0.0, 0.0
    log = []
    start = last = time.time()
    print("Overlord is watching. Press Ctrl+C to stop.")
    try:
        while True:
            now = time.time()
            dt, last = now - last, now
```

```
            title = active_window_title()
            site = match_distraction(title)
            distracted = site is not None
            truce_left = max(0, math.ceil(truce_until - now))
            if truce_left > 0:
                anger = 0.0
            else:
                anger += (gain if distracted else -decay) * dt
                anger = max(0.0, min(100.0, anger))
            level = level_for(anger)
            text = lcd_text(level, site, truce_left)

            if ser is None:
                shown = title[:40].encode("ascii", "replace").decode()
                print(f"{now - start:6.1f}s  anger {anger:5.1f}  level
{level}  {text:<16}  [{shown}]")
            else:
                try:
                    if exchange(ser, f"{level}|{text}\n"):
                        truce_until = now + truce_len
                        print("Button pressed: truce granted")
                except (serial.SerialException, OSError):
                    print("Lost the Arduino. Reconnecting...")
                    ser = open_port(args.port)

            log.append((round(now - start, 1), round(anger, 1), distracted))
            time.sleep(TICK)
    except KeyboardInterrupt:
        print("\nStopping.")
    finally:
        if ser is not None:
            ser.close()
        save_report(log)


if __name__ == "__main__":
    main()
```

Running it

Start-up steps (put these on the sticky note)

1. Plug the Arduino into the host laptop. Close Serial Monitor.
2. Open PowerShell in the `overlord` folder and run `py overlord.py`.
3. Wait for `Connected on COMx`. About 3 seconds later the LCD changes from WAITING FOR PC to ALL IS CALM.
4. Click a YouTube tab so it is the front window. The mood climbs: level 1 after about 3 seconds, level 2 after about 8, level 3 after about 15.
5. Click back to a document. The creature calms down within 10 to 15 seconds.
6. Press the creature's button for a 15-second truce. It starts within about a second of the press.
7. Press Ctrl+C in PowerShell to stop. It saves `session.csv` and `session_report.png` in the same folder.

Options. `py overlord.py --dry-run` tests the Python side with no Arduino. It prints the window titles it sees, so keep private windows off screen while recording. `--port COM5` picks the port by hand. `--real` switches to slow timing: level 3 after about 60 seconds and a 5-minute truce.

What to edit. The `DISTRACTIONS` keywords, `LEVEL_AT` and the two timing tuples, and the taunts in `lcd_text`. The taunts are Person C's job: keep each line to 16 ASCII characters.

Troubleshooting

When something fails, find which side is broken before you change anything. Three tests split the problem, and whichever one passes is not the culprit.

1. **Self-test sketch:** do the parts and wiring work?
2. **Serial Monitor test** (send `2|HELLO` as described under File 2): does the creature sketch work without Python?
3. **`py overlord.py --dry-run`:** does the Python logic work without the Arduino?

If you are stuck for 15 minutes, change one thing at a time and re-run the same test. After 20 minutes, swap roles or ask an organizer.

Upload and Arduino IDE

Symptom	Likely cause	Fix
No COM port appears under Port	Charge-only USB cable, or a missing driver	Try another cable and USB port. In Device Manager, expand Ports (COM and LPT) and watch for a new entry as you plug in. An Unknown device means a driver is needed: ask an organizer
avrdude: stk500_getsync(): not in sync	Wrong board or port, or something else is holding the port	Choose Arduino Uno and the right port. Close Serial Monitor and any running overlord.py. Unplug anything wired to pins 0 or 1
Access denied, or Could not open COMx	Another program has the port	Close Serial Monitor, the Python script and any other Arduino IDE window. Only one can use the port at a time
Servo.h or LiquidCrystal.h not found	Incomplete IDE install	Reinstall Arduino IDE 2. Both libraries ship with it
Strange compile errors after pasting	Smart quotes or hidden characters picked up while copying through chat or email	Copy again straight from the code block in this doc

Python and the USB link

Symptom	Likely cause	Fix
py is not recognized	Python is not on PATH, or only installed as python	Run `python --version`. If it works, use `python` instead of `py` everywhere
No module named serial	pyserial missing, or the wrong package installed	Run `py -m pip uninstall serial`, then `py -m pip install pyserial`

Symptom	Likely cause	Fix
No Arduino found, with a list of ports	Auto-detect did not recognise the board's name	Find your board in the printed list, then run `py overlord.py --port COM5` using its port
Connected, but the LCD stays on WAITING FOR PC with blue eyes	The Arduino is not receiving: wrong sketch loaded, or wrong port	Re-upload the overlord sketch, then restart Python. The RX light on the Uno should flicker twice a second while Python runs
LCD shows NO SIGNAL and CHECK PYTHON mid-demo	Python stopped, crashed or lost the cable for over 5 seconds	Read the PowerShell window for an error and restart `py overlord.py`. The creature recovers by itself once lines arrive
Python runs but the creature never gets angry	The window title has no keyword, or the browser is not the front window	Run with `--dry-run` and read the title in brackets. Add a word from it to `DISTRACTIONS`, and click the tab so it is the active window
Anger rises too fast or too slow for the video	Timing is set for a demo	Edit the first two numbers of `DEMO_TIMING`, or use `--real`
Script stops right after starting	The Uno was still restarting when the first line arrived	Raise `time.sleep(2.5)` in `open_port` to 4

Servo

Symptom	Likely cause	Fix
Servo does nothing	Loose or wrong wire, or no power	Check orange to D9, red to the 5V rail, brown to the GND rail. Try different breadboard holes and run the self-test

Symptom	Likely cause	Fix
Servo twitches, the LCD flickers, or the board resets when it moves	Voltage dip from the servo's current draw	Put the 100 uF capacitor across the 5V and GND rails near the servo, long leg to 5V. Use a USB port on the laptop itself, not a hub. Lower the `shake` values
Servo hums but does not move	The head or a cable is blocking it	Free the head and keep cardboard and wires clear of the horn
Head points the wrong way	It was mounted at a different angle than assumed	Change `ANGLE_AWAY`, `ANGLE_MID` and `ANGLE_YOU` in the sketch and re-upload
Servo buzzes at one end	It is being pushed past its range	Keep angles between 10 and 170 degrees. The sketch already clamps to that

RGB LED and buzzer

Symptom	Likely cause	Fix
LED stays dark	Longest leg not on GND, or two legs sharing a column	Check the longest leg goes to the GND rail and each colour leg has its own column and resistor
Colours inverted (calm is bright, furious is off)	Common-anode LED	Wire the longest leg to 5V instead of GND, set `COMMON_ANODE = true`, and re-upload
Colours in the wrong order	Legs swapped	Swap the two jumper wires, or swap the pin numbers in the sketch
Buzzer is silent	Seal still on, wrong leg, wrong pin, or `SOUND_ON = false`	Peel the sticker off. Put the + leg on the resistor going to D10. Check `SOUND_ON`

Symptom	Likely cause	Fix
Buzzer only clicks	Probably the passive buzzer, which usually shows a green circuit board underneath	Use the active buzzer, the black one with the sticker

Button and LCD

Symptom	Likely cause	Fix
Button always reads YES in the self-test	Button sits the wrong way on the breadboard	Turn it 90 degrees
Truce triggers by itself	Loose wire on A0	Re-seat the wire and keep it away from the servo wires
Truce feels slow	Python reads the button once per half second and applies the truce on the next line	Up to a second of delay is normal. Press, then count one
LCD backlight on but no text, or only blocks	Contrast set too high or too low	Turn the potentiometer slowly, and check LCD pin 3 goes to its middle leg
LCD completely dark	No power, or the pins are shifted one column	Check VDD to 5V, VSS to GND, A through a resistor to 5V, K to GND, and that pin 1 is at the VSS end
Garbage characters, or text on only one row	A data wire (D4 to D7) is loose or swapped, or RW is not on GND	Compare each wire with the LCD table: RS 12, E 11, D4 8, D5 7, D6 4, D7 2

Demo day

Symptom	Likely cause	Fix
A pop-up steals the front window	Notifications	Turn on Do Not Disturb, and close Teams, Discord and email
Laptop sleeps or locks during recording	Power settings	Plug in the charger and set screen and sleep to Never

Symptom	Likely cause	Fix
Creature reacts to the wrong window	A keyword also appears in another window's title	Close other windows while recording, or edit `DISTRACTIONS`
A site is not recognised (Reddit, for example)	The page title may not contain the site's name	Use YouTube for the demo, or run `--dry-run` and add a word from the real title

Fallback plan

A simple creature that works beats an ambitious one that does not, so every failure below has a smaller demo you can still record.

If this fails	Show instead	Decide by
LCD will not light or show text	Run without it. The sketch works with the LCD unplugged, and the head, eyes and buzzer carry the demo	3:30 pm
Servo jams or will not move	Keep the LCD, eyes and buzzer. The creature becomes a stationary watcher whose colours and messages show its mood	3:30 pm
Button is unreliable	Leave the truce out of the video and mention it as a feature	3:30 pm
Window detection fails on the host laptop	Drive the creature by hand from Serial Monitor (see File 2) and say so in the video	3:30 pm
pyserial will not install, or Windows blocks Python	Same as above: Serial Monitor needs no Python	3:30 pm
The Uno stops responding	Ask an organizer for a spare Uno and re-upload the sketch from the second laptop	As soon as it happens

When driving by hand, the four lines are `0|GOOD HUMAN`, `1|YOUTUBE?`, `2|YOUTUBE AGAIN?` and `3|CLOSE IT. NOW.`, sent with the line ending set to Newline.

Always do

- At M1 (1:15 pm), record a 30-second safety video of the simplest version that works, and keep it in the submission folder.
- Photograph the wiring at M1 and M2. If a jumper falls out later, the photo shows where it went.
- Keep a copy of every working sketch and script on a second laptop.
- If something is still unreliable at 4:30 pm, record with the stable parts only.

Show only what is real. If a take uses a typed command or a scripted moment instead of the live system, say so in the video. Judges forgive a limit that was stated, and they notice one that was hidden.

Demo video

The strongest video is one continuous shot of the creature and the laptop screen together, because it proves the reaction is live. Aim for about 90 seconds, and check the organizers' maximum length first.

One-line pitch: Desk Overlord is a desk creature that turns your attention into something you can see. It reacts to the window you are looking at, runs entirely on your laptop, and saves a private session report.

Time	What the camera shows	What C says
0:00-0:08	The creature facing away, eyes green, laptop showing a document	"This is Desk Overlord. It keeps an eye on your screen so you can keep your eyes on your work."
0:08-0:20	A hand clicks a YouTube tab. Eyes turn amber and the head turns	"Open a distraction and it notices. First it just watches."
0:20-0:45	Level 2, then level 3: orange eyes, shaking head, red flashing, beeping	"Stay on it and it gets angry. By fifteen seconds it is shaking, flashing and beeping." (Stay quiet during the fast beeping.)
0:45-0:55	The tab is closed. The creature calms and the eyes go green	"Close the tab and it calms down."

Time	What the camera shows	What C says
0:55-1:05	A finger presses the button. The LCD shows TRUCE	"Beg for mercy with the button and you get fifteen seconds of peace."
1:05-1:20	A slide or hand-drawn diagram: laptop, USB cable, Arduino	"A Python script on the laptop reads the window title and runs a simple anger score. An Arduino drives the servo, lights, buzzer and screen. Nothing leaves the laptop, and there is no machine learning, on purpose."
1:20-1:30	The session report chart	"It saves a chart of the session. We built and tested it today. We have not yet tested whether it improves focus, and that is our next step."

Filming checklist

- Phone in landscape on a stack of books or a tripod, Do Not Disturb on, desk cleared, light on the creature's face.
- Record 3 full takes and pick the best. The buzzer is loud, so keep the level 3 moment to a few seconds.
- Before the take: plug in the laptop, close other apps, turn off notifications, and run once with the real tabs you will use.
- Optional: screen-record the laptop (Windows key + Alt + R records the front window with Xbox Game Bar) and cut to it for the session chart. Skip the edit if time is short.
- Save the best take in two places, upload it, and open the link on another device before you call it done.

Judge Q&A

Intel judges and professors will test whether you understand what you built. Say what the system does, say what it does not do, and never claim more than the test showed.

Claims you can make, and claims you cannot

You can say	You cannot say
It reacts to the active window on a live laptop, and the code checks passed in simulation	It improves focus or productivity. You have no evidence yet
Nothing leaves the laptop, and the session file holds no window titles	It is secure or private by design beyond that. You have not audited it
It is rule-based, so it is explainable and works offline	It uses AI or machine learning. It does not
It recovers if the cable is unplugged or Python stops	It is ready for real users

Likely questions

Question	Honest answer
Why not just use a focus app?	Apps live on the same screen as the distraction and are easy to dismiss. A physical object in the room adds a social cue you cannot close with a click. We have not tested whether that works better, and a comparison is the next experiment
Is this AI?	No. It is a keyword match and a scored state machine, chosen because it is explainable, predictable and runs offline. A classifier would add error we could not explain in a live demo
What about privacy?	The script reads only the title of the active window, on the same laptop, and nothing is sent anywhere. The saved session file records only time, score and a yes or no per check. Window titles stay on screen and in memory only. We have not done a security review

Question	Honest answer
What happens if the Python script crashes?	The Arduino notices after 5 seconds without a message, shows NO SIGNAL and glows blue. It recovers by itself when messages resume. If the cable is pulled, Python tries to reconnect
Why 9600 baud and plain text?	The messages are tiny, a few bytes twice a second. Plain text can be tested by typing it in Serial Monitor, which is how we tested the creature before connecting Python
How would you know if it worked?	Run the same task for 30 minutes with and without it, measure time on distracting windows, and repeat with more people. Three teammates would not prove anything
What would break it?	Keywords that are missing or appear in a different window's title, a browser with several tabs where the active tab changes, and users who hold the work on a second device. It cannot see content, only titles
What did you build today, and what is simulated?	The wiring, both sketches and the Python script, built today. The logic was run against simulated input for testing. List here anything in the video that was staged
Could this be misused for surveillance?	Yes, in principle: a version on someone else's laptop could track their habits. This one shows its state openly, keeps no titles, and asks the user to start it. A real product would need explicit consent and clear controls
What is next?	Whitelist mode for work apps, per-user keyword learning, a multi-person leaderboard, a controlled comparison, and a quieter alert option

Rehearse once: C asks the questions, A and B answer in turns, and the team keeps each answer to 20 seconds. Anyone can say "We haven't tested that yet" without it hurting the score.